Phase 1 — Razorpay Integration Audit (BBPS Scope) — eRupaiya Flutter App

Audit date: 30-Oct-2026

Type: READ-ONLY audit (kooni hi application code modify kela nahi)

Scope: Fakt BBPS (Bharat Bill Payment System) modules madhe vaparlyala jaanaar Razorpay flow. Non-BBPS flows (udahararth Digital Gold) detail analysis kaadhe takle ahet, punhi mahutvachi findings sangitli ahet.

---

1. Project Razorpay Implementation Overview

- **Package:** `razorpay_flutter: ^1.1.1` (`pubspec.yaml` line 15).
- **Single shared wrapper:** Puriva app madhe Razorpay checkout **ekach class** madhun open hoto — `RazorpayService` (`lib/features/paymentgateway/razorpay_service.dart`). Hi ek **singleton** ahe (`RazorpayService.instance`).
- **Key source:** Razorpay Key ID **Flutter madhe hardcode NAHI**. To mag backend `create-order` API chya response madha `key` field madhun yeto. Mhanje key backend kade ahe ani to runtime la device var pathavto.
- **Key Secret:** Flutter code madhe **konthi hi Razorpay key secret NAH** (search karun verify kela). ✓ Security drushtya hi bari go.
- **Signature verification:** Flutter side la `razorpay_signature` / `razorpay_payment_id` cha **konthi hi handling NAHI**. Payment khara "kela ka nahi" he **fakt backend status API** ne poll karun tharte. (He design badal CCAvenue sathi mahutvachi ahe — PAHI §8.)
- **Kill-switch:** Profile API chya `is_razorpay_disable` flag ne backend remote paasun payments pause karu shakto — `RazorpayGuard` hi check dar payment sathi hoto.
- **Payment chya 4 BBPS invocation points ahet:**

1. **Education Fees** (Tuition/Rent etc.) → `runEducationPayNow`
1. **Mobile Recharge** → `payment_bottom_sheet.dart` (recharge sheet)
1. **BBPS Bill Payment** (electricity, water, gas, DTH etc.) → `payment_bottom_sheet.dart` (biller sheet)
1. **Biller detail / subscription + FastTag direct pay** → `biller_detail_view.dart`

Dar Razorpay payment cha 3-stage pattern (sarva BBPS modules madhe same):

```
Stage 1: POST <module>/create-order   → backend {order_id, key, transaction_ref_id}
Stage 2: RazorpayCheckout open        → user pays → success/failure callback
Stage 3: GET <module>/status/<ref>    → backend polling → SUCCESS/FAILED/PENDING screen
```

Flutter **konthi hi Razorpay order create karat nahi, signature verify karat nahi** — he sagla backend che kaam ahe (backend code repo madhe nahi → `Backend code not available for verification`).

---

2. Complete File Inventory (Razorpay-related)

#	File	Responsibility
1	`lib/features/paymentgateway/razorpay_service.dart`	Razorpay checkout wrapper (singleton) — ekmathra jagya `Razorpay().open()`
2	`lib/features/paymentgateway/razorpay_guard.dart`	Payment-purv chi profile-readiness + kill-switch check
3	`lib/features/educationFees/helpers/education_pay_now.dart`	Education "Pay" handoff (order → checkout → result)
4	`lib/features/educationFees/helpers/education_payment_reconcile.dart`	Gateway-failure nantar single-shot status reconciliation
5	`lib/widgets/processing_overlay.dart`	Processing screens + 60s status polling engine
6	`lib/features/educationFees/services/education_fees_service.dart`	`create-order` + `status` API calls (education)
7	`lib/features/educationFees/repositories/education_fees_repository.dart`	Repository passthrough (education)
8	`lib/features/educationFees/models/education_fees_responses.dart`	Order/status request-response models
9	`lib/features/mobile_prepaid/components/payment_bottom_sheet.dart`	Recharge ani Bill-payment sheets — 2checkout call sites
10	`lib/features/mobile_prepaid/controllers/mobile_prepaid_controller.dart`	Recharge order + status verify + single-flight guard
11	`lib/features/mobile_prepaid/repositories/mobile_prepaid_repository.dart`	`recharge/create-order` + status APIs
12	`lib/features/mobile_prepaid/models/recharge_order_result.dart`	Recharge order response model
13	`lib/features/mobile_prepaid/models/prepaid_transaction_status.dart`	Recharge status model
14	`lib/features/services/controllers/biller_detail_controller.dart`	Bill order + status polling controller
15	`lib/features/services/repositories/biller_repository.dart`	`bill/create-order` + `recharge/status` APIs
16	`lib/features/services/models/service_payment_order_result.dart`	Bill order response model
17	`lib/features/services/models/recharge_status_result.dart`	Bill/recharge status model
18	`lib/features/services/views/biller_detail_view.dart`	Direct Razorpay call (subscription bill/FastTag path)
19	`lib/features/profile/models/profile_model.dart`	`is_razorpay_disable` kill-switch field
20	`lib/services/payment_device_context_service.dart`	Device context (GPS/VPN/root/deviceId) — order request la attach
21	`lib/constants/api_constants.dart`	Sarva endpoints
22	`lib/features/digital_gold/components/gold_payment_summary_sheet.dart`	**Non-BBPS** — punhi ek finding ahe (§9.3)

---

3. CORE FILE — `razorpay_service.dart` (Line-by-Line)

Path: `lib/features/paymentgateway/razorpay_service.dart` (77 lines — sagla file vapale)

Line(s)	Code	Saral Spashtikaran
1	`import 'package:razorpay_flutter/razorpay_flutter.dart';`	Official Razorpay Flutter plugin. Yethun `Razorpay` class ani callback response types yetat.
3–5	`typedef RazorpaySuccessCallback = void Function(String paymentId);` etc.	Tin callbacks che simple function-types: success (paymentId pathavto), failure (message pathavto), external-wallet (wallet che nav pathavto). **Note:** success la fakt `paymentId` yeto — signature NAHI.
7–10	`class RazorpayService { RazorpayService._(); static final instance = ... }`	**Singleton pattern.** Constructor private ahe, mhanje app bhar ekach instance vaparalat. Yacha fayda: ekach `Razorpay` object, duplicate listeners control hotat.
12	`Razorpay? _razorpay;`	Razorpay SDK cha actual object. Nullable — aajun checkout open zala nasel tar null.

14–17	`dispose() { _razorpay?.clear(); _razorpay = null; }`	Listeners clean karnyasaathi helper. ⚠ **Finding:** hi method konthi hi call kelayala **adisla nahi** (grep ne confirm) — practically cleanup `openCheckout()` chya line 36 var hote.
19–29	`Future<void> openCheckout({...})`	**Ekmathra entry point dar payment sathi.** Params: `amount` (rupees madhe), `name`, `description`, `orderId`, `keyOverride` (backend kadeun mileli key), 3 callbacks, `prefill` (contact/email pre-fill).
30–34	`final key = keyOverride.trim(); if (key.isEmpty) { onFailure?.call('Razorpay key is missing.'); return; }`	**Guard:** jar backend ne key pathavli nahi tar checkout open hokh nahi — lagan onFailure call hoto. Yane app crash tala ani user la spast message yeto.
36–37	`_razorpay?.clear(); _razorpay = Razorpay();`	**Dar checkout purvi eski listeners havgatli jaat.** Yacha arth: jar 2 veli payment kele, tar 1adya che listeners 2rya la duplicate fire **karu shakat nahi**. He duplicate-callback prevention cha 1st layer ahe.
39–44	`on(EVENT_PAYMENT_SUCCESS, (PaymentSuccessResponse response) => onSuccess?.call(response.paymentId ?? ''))`	Razorpay SDK la success listener register karto. User che payment gateway kade success zale ki hi event fire hote; `paymentId` (udah. `pay_XXXX`) callback madhun forward hoto. **`paymentId` fakt UI/record sathi vaparala jaato — backend la kathyahi pathavlat nahi** (he §8 madhe detail).
45–50	`on(EVENT_PAYMENT_ERROR, ...)`	Payment fail/cancel zalyavar hi event. `response.message` (gateway cha message) forward hoto. **User cancel pan yaahich yeto** — vegla "cancel" event implement kelea nahi.
51–56	`on(EVENT_EXTERNAL_WALLET, ...)`	Jar user external wallet (udah. Paytm app) nividato tar hi event — wallet che nav yeto. Amchya flows madhe he pan "verify and show" kade jaate.
58–59	`normalizedAmountPaise = amount > 0 && amount < 1 ? 100 : (amount * 100).round()`	Razorpay la amount **paise** madha havsa. ₹2000 → 200000. Edge case: ₹1 peksha kami amount asel tar tőí 100 paise (₹1) mantat — negative/shunya payment tala.
61–72	`options = {'key': key, if (orderId...) 'order_id': orderId, 'amount': ..., 'name': ..., 'description': ..., 'prefill': ..., 'theme': {'color': '#F37021'}}`	Checkout config. `order_id` asel tar **backend-created order** la attach hote (recommended flow); nasel tar plain checkout. `theme #F37021` = app cha orange brand color.
74	`_razorpay!.open(options);`	**Razorpay Checkout UI open hote** — he native (Android/iOS) gateway screen ahe, amcha Flutter UI nahi. User card/UPI/wallet nikalta.

4. CORE FILE — `razorpay_guard.dart` (Line-by-Line)

Path: `lib/features/paymentgateway/razorpay_guard.dart` (52 lines)

Line(s)	Code	Spashtikaran
7–10	`pausedMessage / loadingMessage`	Do static user-facing messages.
12–15	`isPaused(ref) => profile?.isRazorpayDisabled ?? false`	Profile state madhla kill-switch read karto. Profile null asel tar default `false` (payments chalu).
17–41	`ensureProfileReadyAndNotPaused(ref)`	**Payment sathi chi dar gate.** (1) Profile load nasel tar `fetchProfileIfNeeded()` ne API hunun aanat; (2) toih pan null tar "profile load hona" snackbar + `false`; (3) `isRazorpayDisabled == true` tar "Payments are paused" snackbar + `false`; (4) sagla ok tar `true`.
43–50	`ensureNotPaused(ref)`	Same check, pun profile fetch agahan (sync version).

Kahate ahe he: Backend `is_razorpay_disable=true` set karun **app update vangarich sagle Razorpay payments band** karu shakto. Hi emergency kill-switch ahe. CCAvenue sathi pan equivalent pahije.

5. BBPS MODULE 1 — Education Fees Flow

5.1 Entry points

- `lib/features/educationFees/views/education_fees_payment_view.dart:95` → `onPayNow: (payable) => runEducationPayNow(...)`
- `lib/features/educationFees/views/education_tutor_profile_view.dart:101` → same helper (donhi jagi ekach behaviour).

5.2 `education_pay_now.dart` (121 lines) — line-by-line

Line(s)	Kay hote
30	`RazorpayGuard.ensureProfileReadyAndNotPaused(ref)` — payment purvi kill-switch + profile check. Fail tar **return** (checkout open hokh nahi).
34–36	`_firstNonEmpty(...)` — recipient name/account/IFSC: explicit param (tutor-profile screen kadeun) asel to, nahitar shared controller state.
38–54	`repository.createOrder(...)` — **POST `/api/education/create-order`**. Exception aailya tar red snackbar "Failed to create order" + return.
56–65	**Response validation guard:** `order.status != true` kinva `orderId` khali kinva `key` khali → backend cha `message` dakhun payment **thambvto**. Yane invalid order ne checkout open hokh nahi.
69	`var navigationStarted = false;`	**Duplicate-navigation guard (2nd layer).** Gateway kadeun success ani failure donhi fire zale, kinva user already dusrya screen var gasla, tar dusrya callback madhe kaam hokh nahi — **ekach payment result flow**.
71–76	`RazorpayService.instance.openCheckout(amount: payable, name..., description: 'Tuition fee payment', orderId: order.orderId, keyOverride: order.key)` — checkout open, **backend order la attach**.
77–88	**onSuccess:** `navigationStarted` check → `openEducationPaymentProcessing({...})` — full-screen processing route var jaat, yethle **60s polling** chalu hote (§5.4). Map madhe: `transactionRefId` (status API sathi), `paymentType`, `recipientName`, `maskedAccount` (`****1234` mask, line 110–115), `fallbackAmount`, `paymentId`.
89–105	**onFailure:** 60s polling **karayala nahi** — gateway ne failure confirm keli ahe. Tadya badala `reconcileEducationPaymentOnFailure(...)` — **single-shot** status check (5.3).

5.3 `education_payment_reconcile.dart` (160 lines)

- L37–48: `showPaymentProcessingWhile(work: repository.fetchPaymentStatus(transactionRefId))` — "Verifying Payment" overlay khali **ekda** status API. Network error tar `timedOut=true` (indeterminate → PENDING).
- L54–72: Status **SUCCESS** aasel tar (durya session madhe payment jhala) → `EducationPaymentThankYouView`.
- L76–80: `resolvedStatus` = timeout/null → `PENDING`; PENDING/PROCESSING → to status; nahitar → `FAILED`.
- L97–151: Local `TransactionHistoryEntry` banavun `context.go(RouteConstants.transactionDetailForStatus(status))` — **result screen** (success/pending/failed transaction detail). L109: `pgTransactionId = paymentId asel tar to, nahitar transactionRefId` — fakt display sathi.
- L117: `method: 'Razorpay / Online'` — hardcode display label.

5.4 Processing + Polling — `processing_overlay.dart`

- L23–49 `openEducationPaymentProcessing`: `navigatorKey.currentContext` vaparun `context.go(paymentProcessing, extra)` — **app kathyahi context gasli ki lifecycle listener ne resume var open hote**. `opened` flag ne duplicate navigation talt.

- L51–52: `_statusRetryInterval = 1s`, `_processingTimeout = paymentProcessingDuration` (60s — `payment_processing_loader.dart` madhe defined).

- L63–110 `showPaymentProcessingWhile<T>`: OverlayEntry insert → `work` future + 60s Timer race → result kinva `timedOut`. Finally overlay remove.

- **L510–714 `PaymentProcessingOverlay` (full-screen, back-locked):**

- L532–714 `useEffect(const [])` madhe `pollStatus()` loop: **dar 1 second la GET `/api/education/status/<ref>`** (repository → service → Dio).

- L674–681: `isSuccess` → ThankYou; `isTerminalStatus` (FAILED kinva REFUND contain) → status screen.

- L682–689: API error (network down) → **loop chalu rahte** (catch karto, fall out hokh nahi) — internet yeta chi wait.

- L696–707: 60s countdown Timerหมดลา → `goToPending()` — PENDING transaction detail screen.

- L716: `PopScope(canPop: false)` — **verifying astana user back dabu shakat nahi**.

- L656–663: `transactionRefId` khali asel tar Transactions screen kade fallback.

5.5 Education API layer

`education_fees_service.dart`:

- **createOrder (L158–207):**

- L167: `PaymentDeviceContextService().collect()` — device context map merge (sikhane 5.6).

- L168–175: `EducationCreateOrderRequest.toJson()` — body: `recipient_name` (String), `account_no` (String — unmasked asel tar te), `ifsc` (String), `amount` (double, 2-decimal). Sarva **required** (constructor `required`).

- L177–190: `fee_type` (optional, String); House/Shop Rent sathi extra `payment_type` pan.

- L191–193: `POST /api/education/create-order` (JSON).

- L197–202: **DioException madhla response body pan parse karto** — mhanje HTTP 400 asla tar pan backend cha `message` user la dakhvta yeel.

- **fetchPaymentStatus (L209–228):** `GET /api/education/status/{transactionRefId}` → `EducationPaymentStatusResponse`.

`education_fees_responses.dart`:

- **`EducationCreateOrderResponse` (L289–327)** — fields: `status` (bool), `message`, `txn_id` (int), `transaction_ref_id` (String — **amcha status-polling key**), `order_id` (String — Razorpay la), `amount`, `currency`, **`key`** (Razorpay public key ID — backend generated).

- **`EducationPaymentStatusResponse` (L329–416)** — `data` nested flatten karto; `payment_status` / `status` donhi keys vahato; getters: `isSuccess/isPending/isProcessing/isFailed` (uppercase compare); amount/service_charge/gst/payable_amount/banner fields transaction detail sathi.

5.6 Device context (PCI-DDA type fields) — `payment_device_context_service.dart`

L43–64: Order request la merge honare fields: `latitude`, `longitude`, `isMock` (mock location), `isVpn`, `isRooted`, `deviceId` (Android ID / iOS IDFV), `timestamp`. 30s cache (L11–35) + in-flight dedupe. **Purpose:** fraud/risk signals — backend/BBPS regulatory requirement. Location permission naasel tar safe-fallback (try/catch).

---

6. BBPS MODULE 2 — Mobile Recharge

6.1 Order create — `mobile_prepaid_controller.dart`

- **`createRechargeOrderWithPlan` (L484–544):**

- L488–493: operator/mobile missing → error, return null.

- L495–499: **Single-flight guard** — `state.isPaymentInFlight` asel tar duplicate create-order **ignore** (log: "Payment already in flight"). He duplicate payment request prevention cha **3rd layer** ahe.

- L501–510: State reset (isRecharging, in-flight=true, eski results null).

- L512–518: repository call; fail tar `isPaymentInFlight=false` + message. **Yad rakha:** `isPaymentInFlight` payment **complete hoen merichet true rahte** (L519 comment) — verification nantar controller reset.

- **`verifyRechargeStatus` (L546–597):** `_fetchRechargeStatusWithProcessingPoll` — PROCESSING asel tar deadline paryant poll; result state madhe bharto (`verifiedTransaction`), success/failure messages set.

6.2 API — `mobile_prepaid_repository.dart`

- **`createRechargeOrder` (L311–354):** `POST /api/recharge/create-order`, **form-urlencoded** body: `mobile` (String), `amount` (String, 2-decimal), `operator` (String), `use_wallet` (int 0/1), `desc` (String), **`idempotency_key`** (client-side UUID v4, L356–369 — duplicate order taltayla). Response → `RechargeOrderResult {isSuccess, message, transaction_ref, txn_id, order_id, key}`.

- **Status APIs:** `GET /api/transaction/status/{id}` (L371–391) ani `GET /api/recharge/status/{id}` (L393–411) → `PrepaidTransactionStatus` (`status/message/amount/operator/mobile/paymentMode/walletAmount/razorpayAmount/transactionId/updatedAt` + isSuccess/Pending/Processing getters). `validateStatus <600` — mhanje 4xx/5xx pan parse hote.

6.3 Checkout — `payment_bottom_sheet.dart` (recharge sheet, L913–1300)

- L1233–1256: Pay button → `createRechargeOrderWithPlan` → order null/`orderId`/`key` khali → snackbar + return.

- L1261–1275: `_startRazorpay(amount: remainingAmount, ..., prefill: {'contact': mobile})` — **eCoins (wallet) apply kelyavar fact BAKI_ROPAY (remaining) Razorpay la jaato**.

- L913–936 `_startRazorpay`: RazorpayGuard check → `openCheckout(description: 'Recharge', ...)`.

- **onSuccess (L937–977):** verification loader khali `verifyRechargeStatus` → `_prepaidOutcome(timedOut, verified)` → `_openPaymentResultFlow(... isMobileRecharge: true)` — Success/Pending/Failure result screen.

- **onFailure (L978–1000+):** `pollIfProcessing: false` ne ekda verify — gateway failure asla tar pan backend ne SUCCESS dila tar to manato (reconciliation-first).

---

7. BBPS MODULE 3 — Bill Payment (Billers) + Module 4 — Biller-detail/FastTag

7.1 Shared sheet — `payment_bottom_sheet.dart` (biller sheet, L155–375)

- L216–283 `_verifyAndShow`: `controller.verifyPayAllServicesStatus(transactionRef, pollIfProcessing: !isGatewayFailure)` → loader khali → outcome mapping (L239–251): SUCCESS→success; timeout/null/PENDING/PROCESSING→pending; gateway-failure kinva FAILED→failure; unknown→**pending (fail-safe)**. Sheet close → `_openPaymentResultFlow`.

- L329–375 `_onPayPressed`: `createPayAllServicesOrder` → order validation (orderId/key/**transactionRef** tinhi havshe) → `remainingAmount <= 0` (full wallet) tar checkout **tula deun direct verify** flow; nahitar `_startRazorpay`.

7.2 Controller — `biller_detail_controller.dart`

- **`createPayAllServicesOrder` (L337–408):** biller/detail/bill null check; **single-flight guard L350–353**; repository call; `isPaymentInFlight` verification paryant true.

- **`verifyPayAllServicesStatus` (L410–440)** + `_fetchStatusWithProcessingPoll` (L69–120): PROCESSING astana deadline (60s) paryant poll; done → in-flight false.

- L334–335 comment: **jun `api/bill/pay` deprecated ahe** — aata create-order + status pattern.

7.3 Repository — `biller_repository.dart`

- **`createPayAllServicesOrder` (L150–218):** `POST /api/bill/create-order` (form-urlencoded). Body: `payment_type`, `biller_name`, `billerid`, `amount` (String), `ref_id`, `fetch_ref_id`, `arr_bill_payment_modes` (comma-joined), `masked_identifier`, `use_wallet` (0/1), **`idempotency_key`** (UUID), `accountHolderName` + `account_holder_name` (donhi keys — backend compatibility), `dueDate` + `due_date`, **device context merge**, ani biller-che dynamic `customer_params` (L179–184). → `ServicePaymentOrderResult {isSuccess, message, transaction_ref, txn_id, order_id, key}`.

- **Status:** `GET /api/recharge/status/{transactionId}` → `RechargeStatusResult` (same model recharge kade share).

7.4 `biller_detail_view.dart` (subscription/FastTag inline path, L1650–1785)

Same order → `openCheckout(description: 'Subscription bill payment', prefill: {contact})` → success/failure/external-wallet donhi `verifyAndShowResult` kade (L1761–1781). FastTag minimum ₹100 client check (L1785–1792). Success zalyavar `EducationPaymentThankYouView` (shared thank-you screen), nahitar `TransactionDetailScreen`.

---

8. Complete API Analysis (Summary Table)

API	Method	Endpoint	Call site	Trigger	Key request fields	Key response fields
Education create order	POST	`/api/education/create-order`	`education_fees_service.dart:191`	Pay tap	recipient_name, account_no, ifsc, amount, fee_type, device-context	status, message, txn_id, transaction_ref_id, order_id, key
Education status	GET	`/api/education/status/{ref}`	`:213` (poll 1s ×60s)	Checkout nantar	path param	payment_status, amount, transaction_id, timestamps, charges
Recharge create order	POST (form)	`/api/recharge/create-order`	`mobile_prepaid_repository.dart:319`	Proceed tap	mobile, amount, operator, use_wallet, desc, idempotency_key	status, transaction_ref, txn_id, order_id, key
Recharge status	GET	`/api/recharge/status/{id}`	`:397`	success/failure nantar	path param	status, razorpayAmount, walletAmount, transaction_id
Generic txn status	GET	`/api/transaction/status/{id}`	`:375`	(prepaid fallback)	path param	vartul
Bill create order	POST (form)	`/api/bill/create-order`	`biller_repository.dart:190`	Pay/Proceed tap	payment_type, billerid, ref_id, fetch_ref_id, customer params, masked_identifier, use_wallet, idempotency_key, device-context	status, transaction_ref, txn_id, order_id, key
Bill status	GET	`/api/recharge/status/{id}`	`:239`	verify phase	path param	status, transaction_id, customer_params, amount_breakdown
Profile (kill-switch)	GET	profile endpoint	profile service	App start / payment purvi	—	**`is_razorpay_disable`**

Sensitive values masked: Razorpay key ID (`rzp_live_████…`) runtime-la response kadeun yeto — repo madhe nahi. Key secret kathyahi nahi.

Security & Verification — Final Verdict

Item	Status	Evidence
Key ID Flutter madhe hardcode?	**NAHI** ✓	Fakt `keyOverride` param, backend `key` field kadeun
Key Secret in app?	**NAHI** ✓	Full-repo grep: kadhihi nahi
Order ID kuthe generate?	**Backend** (create-order response)	Flutter fakt `order_id` options madhe ghalto
Payment ID kutheun?	Razorpay success callback (`response.paymentId`)	`razorpay_service.dart:42`
Signature receive/verify in Flutter?	**NAHI** ✗	`razorpay_signature` konthi code madhe nahi
Verification kuthe hote?	**Backend status API polling** — app kade fakt result check	`processing_overlay.dart:654–694` etc.
Payment ID backend la pathavto?	**NAHI** (fakt display/record)	Konthiya hi status/verify API la paymentId jaat nahi
Webhook/server verification	`Backend code not available for verification`	Flutter repo madhe kadhihi reference nahi
Duplicate prevention	3 layers: `_razorpay?.clear()` (L36), `navigationStarted` flag, `isPaymentInFlight` single-flight + `idempotency_key` (UUID)	§3, §5.2, §6.1, §6.2

---

9. PHASE 1D — Actual Payment Flow Diagram (generic BBPS)

```
[USER] Pay/Proceed tap
   │
   ▼  (Flutter)
RazorpayGuard.ensureProfileReadyAndNotPaused ──fail──► Snackbar, THAMBTO
   │ ok
   ▼
Controller/Helper: create-order ──► [BACKEND] POST .../create-order
   │   (device-context + idempotency_key attach)
   │   ◄── {order_id, key, transaction_ref_id}
   ├─ invalid/empty ──► Red snackbar (backend message), THAMBTO
   ▼
RazorpayService.openCheckout(order_id, key, amount_in_paise)
   │
   ▼  [RAZORPAY NATIVE CHECKOUT — gateway screen]
   ├─ SUCCESS callback (paymentId)
   │    └─► Processing screen / loader
   │          └─ LOOP: GET .../status/{ref} dar 1s, kamya 60s
   │                ├─ SUCCESS    ► ThankYou screen
   │                ├─ FAILED/REFUND ► TransactionDetail(FAILED)
   │                ├─ PENDING/PROCESSING (60s timeout) ► Detail(PENDING)
   │                └─ Network error ► poll chalu rahto (retry)
   ├─ FAILURE/CANCEL callback (message)
   │    ├─ Education: single-shot reconcile (ekda status API) → SUCCESS/PENDING/FAILED screen
   │    └─ Recharge/Bill: verify ekda (pollIfProcessing=false) → outcome screen
   └─ EXTERNAL_WALLET callback → same "verify" path
```

Flutter responsibilities: guard checks, order request, checkout open, callbacks, status polling, result navigation, masking/display.

Backend responsibilities: Razorpay key supply, Razorpay order creation, payment gateway integration, transaction save/update, status API (source of truth), kill-switch flag. *(Backend code not available for verification.)*

9.1 Cancel ani Internet-loss behaviour (Phase 1F Q9/Q10)

- **User cancel:** Razorpay `EVENT_PAYMENT_ERROR` fire karto (vegla cancel event nahi) → `onFailure` → Education: gateway-failure manun 60s poll tala deun **ekda status check** (`education_pay_now.dart:89–105`) → FAILED/PENDING screen. Recharge/Bill: `pollIfProcessing:false` ne ekda verify.

- **Payment madhe internet gamla:** Checkout native screen gateway kade astana app offline zali tar Razorpay SDK failure/deep-link return deto; amchi side la `onFailure` kinva callback hi nahilyas `navigationStarted` false rahto. **Jala payment backend ver SUCCESS asel tar** user app punha open karun history/status madhun to pahhu shakte; current code madhe **app-return var pending-reconcile screen NAHI** (finding §10).

- **Polling madhe internet gamla:** `pollStatus` catch karto ani loop chalu rahte (L682–689); 60s zale tar **PENDING** screen — wrong FAILED dakhvle jaat nahi (fail-safe).

---

10. PHASE 1E — BBPS Module Inventory

Module	Screen(s)	Razorpay invocation	Controller	Repository
Education Fees (Tuition, House/Shop Rent, School/College Fees)	`education_fees_payment_view.dart:95`, `education_tutor_profile_view.dart:101`	`education_pay_now.dart:71`	`EducationFeesController` (+ `educationFeesRepositoryProvider`)	`education_fees_repository.dart` → `education_fees_service.dart`
Mobile Recharge	`payment_bottom_sheet.dart` (recharge sheet)	`:930`	`mobile_prepaid_controller.dart:484/546`	`mobile_prepaid_repository.dart:311/371/393`
BBPS Bill Payment (electricity/water/gas/DTH etc.)	`payment_bottom_sheet.dart` (biller sheet)	`:293`	`biller_detail_controller.dart:337/410`	`biller_repository.dart:150/235`
Biller detail direct pay / FastTag / Credit-card bill	`biller_detail_view.dart:1749`	inline `openCheckout`	`biller_detail_controller`	`biller_repository`
(Non-BBPS) Digital Gold	`gold_payment_summary_sheet.dart:269`	`keyOverride: ''`	—	—

§10 Unclear / Missing items (findings)

1. **Signature verification client kade nahi** — backend kade ahe ka te `Backend code not available for verification`. CCAvenue migration sathi he confirm karave.
1. **`paymentId` backend la jaat nahi** — gateway transaction ani backend transaction cha link fakt `transaction_ref_id` ne; Razorpay cha payment record match karnyala backend webhook/own store asel ka — confirm pahije.
1. **Digital Gold `keyOverride: ''` (L273)** — empty key guard mule `onFailure('Razorpay key is missing.')` **nekhamich** yeto; punha nantar buy API call hoto. He BBPS scope baher ahe, punhi **suspicious/dead path** ahe — alageedel team la sangave.
1. **`RazorpayService.dispose()` (L14–17) konthi call karat nahi** — memory leak chance kami (singleton), pun cleanup gap ahe.
1. **Cancel = Payment_ERROR** — "user cancelled" ani "genuine failure" madhla farak message string varun mant nahi; vegla cancel handling implement kelea nahi.
1. **Education create-order madhe `idempotency_key` NAHI** (recharge/bill madhe ahe) — double-tap protection fakt `isPaymentInFlight`-var (education madhe tohi guard nahi — `runEducationPayNow` stateless function ahe). **Double Pay Now tap** veli donhi order API call hoohu shakte — Phase 2 madhe pahave.
1. External-wallet callback: saglyaa kade fakt verify path ahe — wallet deep-link complete karnyacha extra kaam nahī.

---

11. Conclusions & CCAvenue-Readiness Notes

1. **Architecture clean ahe:** Ek wrapper (`RazorpayService`), ek guard (`RazorpayGuard`), per-module "create-order → checkout → status-poll" pattern. CCAvenue sathi **same pattern tiho theun fakt wrapper badalne** shakte — modules la spast seam ahe.
1. **Verification model = backend status polling** (signature nahi). Mhanje gateway badalalyavar pan Flutter side flow toach rahel; fakt order/result APIs badaltat.
1. **Kill-switch, device-context, idempotency (2/3 modules)** already ahet — he BBPS compliance assets ahet.
1. **Pahila module analysis sathi recommendation: Education Fees** — karan (a) to saglyat naveen ani fully-shared flow ahe (`runEducationPayNow` ekach helper), (b) polling/reconcile donhi patterns yethe ahet, (c) idempotency gap yethe disla — CCAvenue design madhe to fix hoohu shakte.

---

12. Questions for Backend Team

1. Razorpay signature verification backend kade hote ka (webhook/SDK)? Kontha endpoint var?
1. `create-order` madhla `key` test/test-mode ahe ka live — environment switch kasa hote?
1. `transaction_ref_id` ani Razorpay `payment_id` cha mapping backend kade store hote ka?
1. Payment pending astana **Razorpay webhook → status update** hote ka? (App la fakt status API diste.)
1. `is_razorpay_disable` flag CCAvenue sathi pan common rahel ka vegla flag havsa?
1. Education flow la `idempotency_key` support ahe ka? (Recharge/Bill madhe client pathavto.)

Report generated by read-only audit. Application source files unchanged (git status verified at audit time).